

Tarea Corta 05: Capítulos de *Little Book of Semaphores*

https://github.com/sergiopb06/Tareas_Paralela_2026.git

Sergio Peralta Barboza

Escuela de Ciencias de la Computación e Informática, Universidad de Costa Rica

CI-0117 Programación Paralela y Concurrente, Grupo 01

Ing. Sleyter Angulo, M.Sc

Jueves 8 de octubre de 2026

Readers-writers problem

El readers-writers problem ocurre cuando una estructura de datos compartida es leída y modificada por hilos concurrentes. Mientras se escribe, hay que impedir que otros hilos puedan acceder a la estructura, ya que podrían leer datos inconsistentes o erróneos. El problema indica que cualquier cantidad de lectores puede estar en la sección crítica del programa a la vez. Mientras los escritores deben tener acceso exclusivo a la sección crítica, es decir, solo puede haber un escritor en la estructura, nadie más. Esto se llama exclusión mutua categórica; la presencia de una categoría (lectores) excluye a otra (escritores), pero no a sus propios miembros.

Para la solución a este problema se propone lo siguiente:

Writers solution

```
roomEmpty.wait()
    critical section for writers
roomEmpty.signal()
```

Este es el código para los escritores; significa que si la sala está vacía, el escritor puede entrar y excluir a todos los otros hilos de entrar; al salir, se asegura de que la sala está vacía. Para los lectores:

Readers solution

```
mutex.wait()
    readers += 1
    if readers == 1:
        roomEmpty.wait()      # first in locks
mutex.signal()

# critical section for readers

mutex.wait()
    readers -= 1
    if readers == 0:
        roomEmpty.signal()   # last out unlocks
mutex.signal()
```

Este código funciona de la siguiente forma: cuando un lector quiere entrar, se verifica si es el primero; si lo es, señala que el cuarto no está vacío. Sin embargo, igualmente permite que otros lectores sigan entrando. Conforme salen, si un lector es el último en salir, señala que el cuarto ahora está vacío. Si el lector intenta entrar mientras hay un escritor en el cuarto, espera en el semáforo del cuarto mientras tiene el mutex. Esto significa que todos los otros lectores que quieran entrar van a esperar en el mutex, no en el semáforo. El capítulo explica que este patrón es común y que se puede empaquetar en una clase *Lightswitch*: el primero que entra prende las luces, el último que sale las apaga:

Lightswitch definition

```
class Lightswitch:
    def __init__(self):
        self.counter = 0
        self.mutex = Semaphore(1)

    def lock(self, semaphore):
        self.mutex.wait()
        self.counter += 1
        if self.counter == 1:
            semaphore.wait()
        self.mutex.signal()

    def unlock(self, semaphore):
        self.mutex.wait()
        self.counter -= 1
        if self.counter == 0:
            semaphore.signal()
        self.mutex.signal()
```

Con esta clase, el código queda más simple:

Readers-writers initialization

```
readLightswitch = Lightswitch()
roomEmpty = Semaphore(1)
```

`readLightswitch` is a shared `Lightswitch` object whose counter is initially zero.

Readers-writers solution (reader)

```
readLightswitch.lock(roomEmpty)
# critical section
readLightswitch.unlock(roomEmpty)
```

Esta solución presenta un problema, *starvation*: si un escritor llega mientras hay lectores en cola, mientras sigan llegando lectores antes que el último se vaya, el escritor nunca logra entrar a la cola, haciendo que espere indefinidamente para escribir. Por eso se presenta el problema de *No-starve*, para modificar el código y que el escritor no sufra de *starvation*.

No-starve readers-writers

Para la solución de este problema se utiliza una barra de acceso (torniquete). Cuando un lector llega, si el torniquete está abierto, sigue con normalidad y entra a la sección crítica. Si un

escritor llega en la cola, cierra el torniquete y espera a que el cuarto esté vacío. El escritor espera a que todos los lectores que estén dentro del cuarto terminen su trabajo, y cuando lo hacen, entra al cuarto. Los nuevos lectores que llegan esperan su turno hasta que se abra el torniquete, el cual es abierto por el escritor una vez termine su trabajo y salga del cuarto. Esto asegura que el escritor avance su turno y que los lectores esperen mientras lo hace. Los lectores no tocan el torniquete, solo pasan por él. La solución propuesta es la siguiente:

No-starve writer solution

```
turnstile.wait()
    roomEmpty.wait()
    # critical section for writers
turnstile.signal()

roomEmpty.signal()
```

No-starve reader solution

```
turnstile.wait()
turnstile.signal()

readSwitch.lock(roomEmpty)
    # critical section for readers
readSwitch.unlock(roomEmpty)
```

Sin embargo, esta solución solo resuelve que al menos un escritor avance en la cola. Pero no garantiza que todos los escritores avancen; cuando el escritor termina, es decisión del planificador quién despierta; un lector puede colarse delante de un escritor. Por eso hay que crear una solución que le dé prioridad a los escritores.

Writer-priority readers-writers

Para la solución de este problema, se reemplaza el semáforo de la habitación y el torniquete por un semáforo que indica cuándo hay lectores y otro de cuándo hay escritores. El lector pasa por *noReaders*, igual que en el torniquete, y mientras tiene *noReaders*, si es el primero, agarra *noWriters*. Esto garantiza que, mientras haya lectores dentro de la habitación, no pueden entrar escritores. El escritor primero agarra *noReaders*; esto hace que, cuando llega un escritor, bloquee que entren los lectores hasta que todos los escritores terminen su trabajo. Los demás escritores que lleguen no tocan el semáforo de los lectores, van esperando su turno para hacer su trabajo y, cuando ya no hay escritores en la cola, liberan *noReaders* y permiten a los lectores avanzar. La solución propuesta es:

Writer-priority readers-writers initialization

```
readSwitch = Lightswitch()  
writeSwitch = Lightswitch()  
noReaders = Semaphore(1)  
noWriters = Semaphore(1)
```

Writer-priority reader solution

```
noReaders.wait()  
    readSwitch.lock(noWriters)  
noReaders.signal()  
  
    # critical section for readers  
  
readSwitch.unlock(noWriters)
```

Writer-priority writer solution

```
writeSwitch.lock(noReaders)  
    noWriters.wait()  
        # critical section for writers  
    noWriters.signal()  
writeSwitch.unlock(noReaders)
```

Esta solución crea el problema inverso; ahora los lectores pueden sufrir de *starvation*. La implementación que se use depende de la aplicación.

No-starve mutex

En la sección anterior, el problema era *categorical starvation*. Una categoría (lectores) dejaba sin turno a otra (escritores). Ahora el problema es *thread starvation*, que un hilo espere indefinidamente mientras otros avanzan. Para resolverlo se exige *bounded waiting*; el tiempo que un hilo espera en un semáforo debe ser demostrablemente finito. Para demostrar que no existe *starvation*, se realizan las siguientes suposiciones sobre quién decide el orden del hilo:

1. Si solo hay un hilo listo para correr, el planificador debe dejarlo correr.
2. Si un hilo está listo para correr, el tiempo que espera para hacerlo está acotado.
3. Si hay hilos esperando en un semáforo y alguien hace signal, uno de los que esperan debe despertar.
4. Si un hilo espera en un semáforo, el número de hilos que despertarán antes que él está acotado.

Los semáforos que tienen la propiedad cuatro se llaman semáforo fuerte (*strong*) y los que tienen solo la propiedad tres se llaman semáforo débil (*weak*). Para resolver una conjetura de Dijkstra, que propuso que no se puede resolver la exclusión mutua sin *starvation* usando solo semáforos débiles. En 1979, J.M. Morris lo refutó suponiendo un número finito de hilos.

Su algoritmo usa dos torniquetes (t1 y t2) que forman dos salas de espera antes de la sección crítica; funciona en dos fases. En la primera, t1 está abierto y t2 cerrado, así que los hilos se acumulan en la segunda sala. En la segunda, t1 se cierra y t2 se abre; los hilos acumulados pasan a la sección crítica. Los contadores room1 y room2 indican cuántos hilos hay en cada sala y un mutex protege room1. El último hilo en cruzar la sala 1 abre t2; el último en salir de la sala 2 lo vuelve a abrir. La solución descrita es la siguiente:

Morris's algorithm

```
mutex.wait()
    room1 += 1
mutex.signal()

t1.wait()
    room2 += 1
    mutex.wait()
    room1 -= 1

    if room1 == 0:
        mutex.signal()
        t2.signal()
    else:
        mutex.signal()
        t1.signal()

t2.wait()
    room2 -= 1

    # critical section

    if room2 == 0:
        t1.signal()
    else:
        t2.signal()
```

Cigarette Smokers problem

El siguiente caso, *Cigarette Smokers Problem*, fue presentado por Suhas Patil. El problema dice que hay cuatro hilos: un agente y tres fumadores. Para fumar, un fumador necesita tres ingredientes: tabaco, papel y fósforos. Se asume que:

- Cada fumador tiene una provisión infinita de un solo ingrediente (uno tiene tabaco, otro papel,

otro fosforos).

- El agente tiene provisiones infinitas de los tres. Repetidamente, elige dos ingredientes distintos al azar y los pone en la mesa.
- El fumador que tiene el ingrediente que falta debe tomar los dos de la mesa, hacer el cigarro, fumar y avisarle al agente para que ponga otros dos.

El agente representa un sistema operativo que asigna recursos; los fumadores, aplicaciones que necesitan dichos recursos. Existen tres versiones del problema, pero se resolverá la versión interesante, que dice que no se puede modificar el código del agente, el cual es:

Agent semaphores

```
agentSem = Semaphore(1)
tobacco = Semaphore(0)
paper = Semaphore(0)
match = Semaphore(0)
```

El agente son tres hilos (A, B, C) que esperan en el semáforo de *agentSem*. Cada vez que se hace *signal* de este, uno de los tres despierta y pone dos ingredientes en la mesa utilizando los semáforos de ingredientes.

Agent A code

```
agentSem.wait()
tobacco.signal()
paper.signal()
```

Agent B code

```
agentSem.wait()
paper.signal()
match.signal()
```

Agent C code

```
agentSem.wait()
tobacco.signal()
match.signal()
```

Sin embargo, la solución natural no funciona, la cual se basa en que cada fumador espere sus dos ingredientes. Falla por un *deadlock*. Por ejemplo, si el agente pone tabaco y papel, el fumador con fósforos espera tabaco y papel; puede avanzar. Pero el fumador con tabaco espera papel; también puede despertar. Un fumador toma tabaco y otro papel. Entonces, el primero

queda esperando el papel y el segundo queda esperando fósforos, que nunca van a llegar; se crea un *deadlock*.

La solución que propuso Parnas consiste en agregar tres hilos auxiliares, pusher, cada uno asociado a un ingrediente. Cuando el agente pone un ingrediente, el pusher correspondiente revisa con un *mutex* si el otro ingrediente que completa el par ya está en la mesa. Si es así, señala al fumador adecuado y limpia la variable booleana; si no, solo registra que su ingrediente está disponible. Los fumadores ahora solo esperan por un único semáforo, que solo se activa cuando los dos ingredientes que necesita están listos:

Smokers problem hint

```
isTobacco = isPaper = isMatch = False
tobaccoSem = Semaphore(0)
paperSem = Semaphore(0)
matchSem = Semaphore(0)
```

Pusher A

```
tobacco.wait()
mutex.wait()
    if isPaper:
        isPaper = False
        matchSem.signal()
    elif isMatch:
        isMatch = False
        paperSem.signal()
    else:
        isTobacco = True
mutex.signal()
```

Smoker with tobacco

```
tobaccoSem.wait()
makeCigarette()
agentSem.signal()
smoke()
```

Generalized Smokers problem

En la versión anterior, el agente ponía dos ingredientes en la mesa y esperaba a que un fumador terminara antes de poner otros. Eso garantizaba que en la mesa nunca hubiera más de un par. Parnas propone quitar esa espera: el agente sigue poniendo ingredientes sin esperar a nadie. Ahora puede haber repeticiones de los ingredientes que hay en la mesa. Como el agente ya no

espera, *agentSem* ya no se usa. Además, se dejan de usar valores booleanos para los ingredientes; ahora se cuentan las unidades.

Generalized Smokers problem hint

```
numTobacco = numPaper = numMatch = 0
```

El resto de la estructura se mantiene: cada *pusher* espera la señal del agente, toma el *mutex* y revisa el estado. Si con sus ingredientes y los de la mesa se completa la pareja, decrementa el contador del ingrediente y despierta al fumador correspondiente.

Cada vez que un agente corre, es como si creara dos *pushers* con un ingrediente cada uno y los enviara a una habitación con los demás. Con el *mutex*, entran uno a la vez a una habitación donde hay una mesa y tres fumadores dormidos. Cada *pusher* revisa la mesa y, si puede completar la pareja, se lo lleva y despierta al fumador correcto; si no, deja su ingrediente y se va sin despertar a nadie. Se menciona que este es un patrón que aparece en otros problemas, que llaman *scoreboard*. Las variables funcionan para mantener un control del estado del sistema, mientras cada hilo pasa por el *mutex*, revisa el estado (*scoreboard*) y reacciona en consecuencia, sin necesidad de comunicarse con otros hilos. El código modificado para el *pusher A* es:

Pusher A

```
tobacco.wait()
mutex.wait()
    if numPaper:
        numPaper -= 1
        matchSem.signal()
    elif numMatch:
        numMatch -= 1
        paperSem.signal()
    else:
        numTobacco += 1
mutex.signal()
```

Barbershop problem

Este problema fue propuesto por Dijkstra. El problema de la barbería plantea un local con una sala de espera de *n* sillas y un cuarto con la silla del barbero. Si no hay clientes, el barbero duerme; si un cliente llega y todas las sillas están ocupadas, se va; si el barbero está ocupado, pero hay sillas libres, el cliente espera sentado; y si el barbero duerme, el cliente lo despierta. Se dice que los clientes invocan *getHairCut*, que un cliente que llega a una barbería llena invoca *balk* (se va), y que el barbero invoca *cutHair*. Además, que mientras el barbero invoca *cutHair*, exactamente un hilo debe estar invocando *getHairCut*.

Se usa un contador *customers* con el número de clientes en la barbería, protegido por un *mutex*, y un límite de *n*, que son las sillas de espera más la silla del barbero. Además, hay cuatro semáforos que forman dos *rendezvous*: *customer* y *barber* para comenzar el corte, y *customerDone*

y *barberDone* para terminarlo. Los nombres siguen la convención de que *customer.wait()* significa “esperar a un cliente”.

Barbershop hint

```
n = 4
customers = 0
mutex = Semaphore(1)
customer = Semaphore(0)
barber = Semaphore(0)
customerDone = Semaphore(0)
barberDone = Semaphore(0)
```

La solución del cliente es que consulta el marcador (*scoreboard*) bajo el *mutex*; si la barbería está llena, libera el *mutex* y se va; si no, incrementa el contador. Luego avisa al barbero y espera a que lo llame. Después recibe el corte y avisa que terminó, espera la confirmación del barbero y decrementa el contador.

Barbershop solution (customer)

```
mutex.wait()
    if customers == n:
        mutex.signal()
        balk()
    customers += 1
mutex.signal()

customer.signal()
barber.wait()

# getHairCut()

customerDone.signal()
barberDone.wait()

mutex.wait()
    customers -= 1
mutex.signal()
```

La solución del barbero es esperar a un cliente con *customer.wait()*, lo llama con *barber.signal()*, le corta el pelo, espera a que el cliente termine con *customerDone.wait()* y confirma con *barberDone.signal()*. Si hay clientes esperando, la siguiente iteración no duerme.

Barbershop solution (barber)

```
customer.wait()
barber.signal()

# cutHair()

customerDone.wait()
barberDone.signal()
```

FIFO Barbershop

La solución anterior de la barbería no garantiza que los clientes sean atendidos en el orden en que llegan. Hasta n clientes pueden pasar el torniquete, avisar al barbero y quedarse esperando en el mismo semáforo. Cuando el barbero señala, cualquiera de los que espera puede despertar. El problema es modificar la solución para que los clientes sean atendidos en el orden en que llegan. La solución consiste en que, en lugar de que todos los clientes esperen un semáforo compartido, cada cliente tiene su propio semáforo. Al pasar el torniquete, el cliente deja ese semáforo en una cola. Cuando el barbero está listo, saca de la cola el semáforo del primer cliente. De este modo, el barbero es el que decide a quién atender y no el sistema.

FIFO barbershop hint

```
n = 4
customers = 0
mutex = Semaphore(1)
customer = Semaphore(0)
customerDone = Semaphore(0)
barberDone = Semaphore(0)
queue = []
```

El código del cliente crea su semáforo, toma el mutex, verifica la capacidad y, si hay lugar, incrementa el contador y agrega su semáforo a la cola. Después avisa al barbero y espera en su propio semáforo. Cuando es atendido, el resto queda igual.

FIFO barbershop solution (customer)

```
self.sem = Semaphore(0)
mutex.wait()
    if customers == n:
        mutex.signal()
        balk()
    customers += 1
    queue.append(self.sem)
mutex.signal()

customer.signal()
self.sem.wait()

# getHairCut()

customerDone.signal()
barberDone.wait()

mutex.wait()
    customers -= 1
mutex.signal()
```

El código del barbero espera a un cliente con `customer.wait()`, toma el mutex, saca el primer semáforo de la cola, suelta el mutex y lo señala. El resto queda igual.

FIFO barbershop solution (barber)

```
customer.wait()
mutex.wait()
    sem = queue.pop(0)
mutex.signal()

sem.signal()

# cutHair()

customerDone.wait()
barberDone.signal()
```

Hilzer's Barbershop problem

Este problema presentado por William Stallings es una versión más complicada del problema de la barbería. La barbería tiene tres sillas, tres barberos y una sala de espera con un sillón para cuatro personas más espacio para esperar parado. La capacidad máxima de clientes esperando

en la tienda es de 20, y quienes lleguen cuando está llena no entran. Los clientes esperan de pie, luego se sientan en el sillón y luego pasan a la silla para el corte de pelo. El que más tiempo lleva en cada etapa es el primero en avanzar, al igual que en *FIFO*. Al terminar, cualquier barbero cobra, pero solo hay una caja, entonces solo se cobra a un cliente a la vez.

Existen las siguientes restricciones:

- Los clientes invocan estas funciones en orden: *enterShop*, *sitOnSofa*, *getHairCut*, *pay*.
- Los barberos invocan: *cutHair* y *acceptPayment*.
- Los clientes no pueden invocar *enterShop* si la tienda está llena.
- Si el sillón está lleno, un cliente que llega no puede invocar *sitOnSofa*.
- Cuando un cliente invoca *getHairCut*, debe haber un barbero concurrente ejecutando *cutHair*, y viceversa.
- Debe ser posible que hasta tres clientes concurrentes ejecuten *getHairCut* y que hasta tres barberos ejecuten *cutHair*.
- Los clientes deben pagar (*pay*) antes de que un barbero pueda *acceptPayment*.
- El barbero debe *acceptPayment* antes de que el cliente pueda irse.

La solución aplica dos veces la técnica *FIFO*: cada cliente tiene un semáforo propio por cada cola, de modo que el barbero decide a quién llamar y se respeta el orden. En la *queue1*, el barbero saca al cliente más antiguo y espera a que el siguiente se siente en el sillón. En el *queue2*, el cliente que lleva más tiempo en el sillón lo manda a la silla. Cada barbero atiende un cliente por vuelta, de modo que hay tres cortes concurrentes. El cliente verifica la capacidad y se mete en *queue1*, avisa y espera su turno, se sienta en el sillón, avisa al barbero, se anota en *queue2*, espera la silla, libera su lugar en el sillón, se corta el pelo y paga. El barbero espera a un cliente de la *queue1*, lo deja pasar al sillón, luego espera a uno de la *queue2*, lo llama a la silla, le corta el pelo y acepta y confirma el pago.

Hilzer's barbershop hint

```
n = 20
customers = 0
mutex = Semaphore(1)
sofa = Semaphore(4)
customer1 = Semaphore(0)
customer2 = Semaphore(0)
barber = Semaphore(0)
payment = Semaphore(0)
receipt = Semaphore(0)
queue1 = []
queue2 = []
```

Hilzer's barbershop solution (customer)

```
self.sem1 = Semaphore(0)
self.sem2 = Semaphore(0)

mutex.wait()
    if customers == n:
        mutex.signal()
        balk()
    customers += 1
    queue1.append(self.sem1)
mutex.signal()

# enterShop()
customer1.signal()
self.sem1.wait()

sofa.wait()
    # sitOnSofa()
    self.sem1.signal()
    mutex.wait()
        queue2.append(self.sem2)
    mutex.signal()
    customer2.signal()
    self.sem2.wait()
sofa.signal()

# sitInBarberChair()

# pay()
payment.signal()
receipt.wait()

mutex.wait()
    customers -= 1
mutex.signal()
```

Hilzer's barbershop solution (barber)

```
customer1.wait()
mutex.wait()
    sem = queue1.pop(0)
    sem.signal()
    sem.wait()
mutex.signal()
sem.signal()

customer2.wait()
mutex.wait()
    sem = queue2.pop(0)
mutex.signal()
sem.signal()

barber.signal()
# cutHair()

payment.wait()
# acceptPayment()
receipt.signal()
```